Examen

Carlos Mario Garcia Alvarez cc: 1036963357

① (a) Tenemos los datos independientes $\{(x_i, y_i)\}_{i=1}^{n}$

Y por ser independiente tenemos entonces que:

$$L(\theta) = \prod_{i=1}^{n} P(y_i \mid x_i ; \theta)$$ Reemplazando Poisson...

$$L(\theta) = \prod_{i=1}^{n} \frac{h\theta(x_i)^{y_i} e^{-h\theta(x_i)}}{y_i!}$$

Como $h_\theta(x_i) = \theta_0 + \theta_1 x_i$

$$L(\theta) = \prod_{i=1}^{n} \frac{(\theta_0 + \theta_1 x_i)^{y_i} e^{-(\theta_0 + \theta_1 x_i)}}{y_i!}$$

(b) Como $l(\theta) = Ln(L(\theta))$

$$\therefore \quad l(\theta) = Ln\left(\prod_{i=1}^{n} \frac{(\theta_0 + \theta_1 x_i)^{y_i} e^{-(\theta_0 + \theta_1 x_i)}}{y_i!} \right)$$

$$l(\theta) = \sum_{i=1}^{n} Ln\left(\frac{(\theta_0 + \theta_1 x_i)^{y_i} e^{-(\theta_0 + \theta_1 x_i)}}{y_i!} \right)$$

$$l(\theta) = \sum_{i=1}^{n} \left(y_i Ln(\theta_0 + \theta_1 x_i) - (\theta_0 + \theta_1 x_i) Ln(e)^{1} - Ln(y_i!) \right)$$

$$l(\theta) = \sum_{i=1}^{n} \left(y_i Ln(\theta_0 + \theta_1 x_i) - (\theta_0 + \theta_1 x_i) - Ln(y_i!) \right)$$

Y como $\theta_0 + \theta_1 x_i > 0$ no hay problema

Powered by CamScanner

y la función de coste asociada en estos casos puede tomarse como $J(\theta) = -\ell(\theta)$

Así

$$J(\theta) = \sum_{i=1}^{h} \left((\theta_0 + \theta_1 x_i) + \ln(y_i!) - y_i \ln(\theta_0 + \theta_1 x_i) \right)$$

(c) Separamos la suma anterior $\quad h_\theta(x_i) = \theta_0 + \theta_1 x_i$

$$J(\theta) = \sum_{i=1}^{h} \left(h_\theta(x_i) - y_i \ln(h_\theta(x_i)) \right) - \underbrace{\sum_{i=1}^{h} \ln(y_i!)}_{\text{No depende de } \theta}$$

Así:

$$J(\theta) = \sum_{i=1}^{h} \left(h_\theta(x_i) - y_i \ln(h_\theta(x_i)) \right)$$

(d) $J(\theta) = \sum_{i=1}^{h} \left((\theta_0 + \theta_1 x_i) - y_i \ln((\theta_0 + \theta_1 x_i)) \right)$

$$\frac{\partial J(\theta)}{\partial \theta_0} = \sum_{i=1}^{h} \frac{\partial}{\partial \theta_0} (\sim) = \sum_{i=1}^{h} \left(1 - \frac{y_i}{(\theta_0 + \theta_1 x_i)} \right)$$

$$\frac{\partial J(\theta)}{\partial \theta_1} = \sum_{i=1}^{h} \frac{\partial}{\partial \theta_1} (\sim) = \sum_{i=1}^{h} \left(x_i - \frac{y_i x_i}{(\theta_0 + \theta_1 x_i)} \right)$$

Así llegamos a que el gradiente es:

$$\nabla J(\theta) = \begin{pmatrix} \partial J/\partial \theta_0 \\ \partial J/\partial \theta_1 \end{pmatrix} = \begin{pmatrix} \sum_{i=1}^{h} \left(1 - \frac{y_i}{(\theta_0 + \theta_1 x_i)} \right) \\ \sum_{i=1}^{h} \left(x_i - \frac{y_i x_i}{(\theta_0 + \theta_1 x_i)} \right) \end{pmatrix}$$

Powered by CamScanner

(e) Interpretaciones físicas:

$h_\theta(x)$ = Esta es la tasa esperada de detecciones, es como la predicción del modelo para el número promedio de detecciones en un intervalo de tiempo fijo.

θ_0 = Es la tasa esperada cuando $X=0$, esto puede considerarse como el ruido propio de l detector.

θ_1 = Significa qué tanto cambia la tasa esperada de detecciones con el cambio de X. Es como una sensibilidad del detector a las condiciones.

---

(2) (a) $$\delta_{i,1}^{[2]} = \frac{\partial J_i}{\partial z_{i,1}^{[2]}} = \frac{\partial \left(\exp(z_{i,1}^{[2]}) - y_i z_{i,1}^{[2]}\right)}{\partial z_{i,1}^{[2]}}$$

$$\Rightarrow \frac{\partial \left(\exp(z_{i,1}^{[2]})\right)}{\partial z_{i,1}^{[2]}} - \frac{\partial \left(y_i z_{i,1}^{[2]}\right)}{\partial z_{i,1}^{[2]}}$$

$$\Rightarrow \delta_{i,1}^{[2]} = \exp\left(z_{i,1}^{[2]}\right) - y_i$$

(b) $$\frac{\partial J_i}{\partial \theta_{1,1}^{[2]}} = \frac{\partial \left(\exp\left(\theta_{1,1}^{[2]} A_{i,1}^{[1]} + b_1^{[2]}\right) - y_i\left(\theta_{1,1}^{[2]} A_{i,1}^{[1]} + b_1^{[2]}\right)\right)}{\partial \theta_{1,1}^{[2]}}$$

$$= A_{i,1}^{[1]} \exp\left(\theta_{1,1}^{[2]} A_{i,1}^{[1]} + b_1^{[2]}\right) - y_i A_{i,1}^{[1]} = \frac{\partial J_i}{\partial \theta_{1,1}^{[2]}}$$

Powered by CamScanner

Por el otro lado

$$\frac{\partial J_i}{\partial b_1^{[2]}} = \frac{\partial\left(\exp\left(\theta_{1,1}^{[2]} A_{i,1}^{[1]} + b_1^{[2]}\right) - y_i\left(\theta_{1,1}^{[2]} A_{i,1}^{[1]} + b_1^{[2]}\right)\right)}{\partial b_1^{[2]}}$$

$$\boxed{\frac{\partial J_i}{\partial b_1^{[2]}} = \exp\left(\theta_{1,1}^{[2]} A_{i,1}^{[1]} + b_1^{[2]}\right) - y_i}$$

(c) $$\delta_{i,1}^{[1]} = \frac{\partial J_i}{\partial z_{i,1}^{[1]}} = \frac{\partial J_i}{\partial z_{i,1}^{[2]}} \cdot \frac{\partial z_{i,1}^{[2]}}{\partial A_{i,1}^{[1]}} \cdot \frac{\partial A_{i,1}^{[1]}}{\partial z_{i,1}^{[1]}}$$

Paso por paso...

$$\frac{\partial J_i}{\partial z_{i,1}^{[2]}} = \exp\left(z_{i,1}^{[2]}\right) - y_i$$ (encontrado en el punto (a))

$$\frac{\partial z_{i,1}^{[2]}}{\partial A_{i,1}^{[1]}} = \theta_{1,1}^{[2]}$$

$$\frac{\partial A_{i,1}^{[1]}}{\partial z_{i,1}^{[1]}} = \sigma'\left(z_{i,1}^{[1]}\right)$$

Así obtenemos $$\boxed{\delta_{i,1}^{[1]} = \left(\exp\left(z_{i,1}^{[2]}\right) - y_i\right) \cdot \theta_{1,1}^{[2]} \cdot \sigma'\left(z_{i,1}^{[1]}\right)}$$

(d) $$\frac{\partial J_i}{\partial \theta_{1,1}^{[1]}} = \underbrace{\frac{\partial J_i}{\partial z_{i,1}^{[1]}}}_{\text{conocido}} \cdot \underbrace{\frac{\partial z_{i,1}^{[1]}}{\partial \theta_{1,1}^{[1]}}}_{x_i} = \left(\exp\left(z_{i,1}^{[2]}\right) - y_i\right) \cdot \theta_{1,1}^{[2]} \sigma'\left(z_{i,1}^{[1]}\right) \cdot x_i$$

del mismo modo

$$\frac{\partial J_i}{\partial b_1^{[1]}} = \underbrace{\frac{\partial J_i}{\partial z_{i,1}^{[1]}}}_{\delta} \cdot \underbrace{\frac{\partial z_{i,1}^{[1]}}{\partial b_1}}_{1} = \left(\exp\left(z_{i,1}^{[2]}\right) - y_i\right)\left(\theta_{1,1}^{[2]}\right)\sigma'\left(z_{i,1}^{[1]}\right)$$

Powered by CamScanner

③ Seq PINN (Physics - Informed Neural Network)

Tenemos que $U_\theta(x,y,t) = \sin(\pi x)\sin(\pi y)\, q_\theta(t)$

Por simplicidad, sea $k(x,y) = \sin(\pi x)\sin(\pi y)$

Así $U_\theta(x,y,t) = k(x,y)\, q_\theta(t)$ .... Así:

$$\frac{\partial^2 U}{\partial t^2} = k(x,y)\, q''_\theta(t)$$

$$\frac{\partial^2 U}{\partial x^2} = q_\theta(t)\cdot(-\pi^2 \sin(\pi x)\cdot\sin(\pi y)) = -\pi^2 k(x,y)\, q_\theta(t)$$

$$\frac{\partial^2 U}{\partial y^2} = -\pi^2 k(x,y) q_\theta(t)$$

Así, sustituyendo en la ecuación de ondas bidimensional

$$k(x,y)\, q''_\theta(t) = -2\pi^2 k(x,y)\, q_\theta(t)$$

$$\therefore \quad q''_\theta(t) = -2\pi^2 q_\theta(t)$$

$$\Rightarrow \boxed{q''_\theta(t) + 2\pi^2 q_\theta(t) = 0}$$

Ahora la idea será calcular las condiciones iniciales.

como $U(x,y,0) = \sin(\pi x)\sin(\pi y) = k(x,y)$

$$\therefore \quad U_\theta(x,y,0) = k(x,y)\, q_\theta(0) \quad \Rightarrow \boxed{q_\theta(0) = 1}$$

$$U_t(x,y,0) = \sqrt{2}\,\pi \sin(\pi x)\sin(\pi y)$$

$$U_t = k(x,y)\, q'_\theta(0) \quad \Rightarrow \boxed{q'_\theta(0) = \sqrt{2}\,\pi}$$

Powered by CamScanner

Así tenemos entonces la siguiente información

$$a''(t) + 2\pi^2 q(t) = 0 \quad ; \quad q(0) = 1 \quad ; \quad q'(0) = \sqrt{2}\pi$$

Ahora la idea es encontrar la solución Analítica.

Supongamos una solución de tipo $q(t) = C_1 \cos(wt) + C_2 \sin(wt)$

donde en este caso $w = \sqrt{2}\pi$

Reemplazando ... $q(t) = C_1 \cos(\sqrt{2}\pi t + C_2 \sin(\sqrt{2}\pi t)$

como $q(0) = 1 \wedge \sin(0) = 1 \Rightarrow \boxed{C_1 = 1}$

como $q'(t) = -C_1 w \sin(wt) + C_2 w \cos(wt)$

$q'(0) = \sqrt{2}\pi \quad \therefore \quad \boxed{C_2 = 1}$

Así encontramos la solución analítica.

$$q(t) = \cos(\sqrt{2}\pi t) + \sin \sqrt{2}\pi t$$

Así la solución de la membrana es la siguiente:

$$U(x, y, t) = \sin(\pi x)\sin(\pi y)[\cos(\sqrt{2}\pi t) + \sin(\sqrt{2}\pi t)]$$

Para el Modelo A

$$q_\theta(t) = \sum_{j=1}^{2} A_j \sin(w_j t + b_j) = A_1 \sin(w_1 t + b_1) + A_2 \sin(w_2 t + b_2)$$

Derivando 2 veces

$$q_\theta''(t) = -A_1 w_1^2 \sin(w_1 t + b_1) - A_2 w_2^2 \sin(w_2 t + b_2)$$

Sustituyendo $\Rightarrow \quad R_A(t) = q_\theta''(t) + 2\pi^2 q_\theta(t)$

Así el residuo Físico es:

$$R_A(t) = A_1(2\pi^2 - w_1^2)\sin(w_1 t + b_1) + A_2(2\pi^2 - w_2^2)\sin(w_2 t + b_2)$$

Powered by CamScanner

Usando el Error cuadratico medio Para la Funcion de Perdida

$$L_{Fisica} = \frac{1}{N}\sum_{i=1}^{N} R_A(t_i)^2$$

con la condicion Inicial de desplazamiento y la de la Velocidad Podemos llegar a que:

$$L_A = \frac{1}{N}\sum_{i=1}^{N} R_A(t_i)^2 + K_1(q_\theta(0)-1)^2 + K_2(q'_\theta(0)-\sqrt{2}\pi)^2$$

con K_1, K_2 constantes

Para el modelo B

$h_1(t) = \sin(w_1 t + b_1)$; $q_\theta(t) = A\sin(w_2 h_1(t) + b_2)$.

$\therefore h_1''(t) = -w_1^2 \sin(w_1 t + b_1)$

$\Rightarrow q''_\theta(t) = Aw_2\left[-w_2 \sin(w_2 h_1 + b_2)(h_1')^2 + \cos(w_2 h_1 + b_2) h_1''\right]$

sustituyendo llegamos a que.

$$q''_\theta(t) = -Aw_2^2 w_1^2 \sin(w_2 h_1(t) + b_2)\cos^2(w_1 t + b_1) - Aw_2 w_1^2 \cos(w_2 h_1(t) + b_2)\sin(w_1 t + b_1).$$

Luego el residuo es $R_B(t) = q''_\theta(t) + 2\pi^2 q_\theta(t)$

$$\therefore R_B(t) = -Aw_2^2 w_1^2 \sin(w_2 h_1 + b_2)\cos^2(w_1 t + b_1) - Aw_2 w_1^2 \cos(w_2 h_1 + b_2)\sin(w_1 t + b_1) + 2\pi^2 A\sin(w_2 h_1 + b_2)$$

y construimos la Funcion de Perdida igual al A.

$$L_B = \frac{1}{N}\sum_{i=1}^{N} R_B(t_i)^2 + K_1(q_\theta(0)-1)^2 + K_2(q'_\theta(0)-\sqrt{2}\pi)^2$$

Powered by CamScanner

(4) Por simplicidad hacemos $K_\mu = \sum_{i=1}^{N} X_{\mu i} q_i$

$$\therefore E(q) = \frac{1}{2}\sum_{i=1}^{N} q_i^2 - \frac{1}{\beta} Ln\left(\sum_{\mu=1}^{P} e^{\beta K_\mu}\right)$$

Calculando las derivadas por partes:

$$\therefore \frac{\partial}{\partial q_i}\left(\frac{1}{2}\sum_{i=1}^{N} q_i^2\right) = q_i$$

$$-\frac{1}{\beta}\frac{1}{\sum_{\mu=1}^{P} e^{\beta K_\mu}} \underbrace{\frac{\partial}{\partial q_i}\left(\sum_{\mu=1}^{P} e^{\beta K_\mu}\right)}$$

$$\sum_{\mu=1}^{P} e^{\beta K_\mu}\, \beta \underbrace{\frac{\partial K_\mu}{\partial q_i}}_{X_{\mu i}}$$

Así llegamos podemos llegar a que:

$$\frac{\partial E(q)}{\partial q_i} = q_i - \frac{\sum_{\mu=1}^{P} X_{\mu i} e^{\beta \sum_{i=1}^{N} X_{\mu i} q_i}}{\sum_{\mu=1}^{P} e^{\beta \sum_{i=1}^{N} X_{\mu i} q_i}}$$

Así el gradiente sería entonces

$$\nabla(E(q)) = \begin{pmatrix} \frac{\partial E}{\partial q_1} \\ \vdots \\ \frac{\partial E}{\partial q_N} \end{pmatrix}$$

que sería expandir con lo que ya encontramos...

Ahora queremos deducir la regla de actualización

$$q_i^{(t+1)} = q_i^{(t)} - \alpha \left.\frac{\partial E}{\partial q_i}\right|_{q = q^{(t)}}$$

$$\therefore q_i^{(t+1)} = q_i^{(t)} - \alpha\left(q_i^{(t)} - \frac{\sum_{\mu=1}^{P} X_{\mu i} e^{\beta \sum_{i=1}^{N} X_{\mu i} q_i^{(t)}}}{\sum_{\mu=1}^{P} e^{\beta \sum_{i=1}^{N} X_{\mu i} q_i^{(t)}}}\right)$$

Powered by CamScanner

$$\therefore \quad q_i^{(t+1)} = (1-\alpha)\, q_i^{(t)} + \alpha \, \frac{\sum_{\mu=1}^{p} x_{\mu i}\, e^{\beta \sum_{i=1}^{N} x_{\mu i} q_i^{(t)}}}{\sum_{\mu=1}^{p} e^{\beta \sum_{i=1}^{N} x_{\mu i} q_i^{(t)}}}$$

Esta sería la regla general de actualización.

Ahora vamos a considerar el caso $\alpha = 1$

$$\therefore \quad q_i^{(t+1)} = (1-1)\, q_i^{(t)} + (1) \cdot \frac{\sum_{\mu=1}^{p} x_{\mu i}\, e^{\beta \sum_{i=1}^{N} x_{\mu i} q_i^{(t)}}}{\sum_{\mu=1}^{p} e^{\beta \sum_{i=1}^{N} x_{\mu i} q_i^{(t)}}}$$

Cuando $\beta \approx 0 \Rightarrow \exp \sim 1$

entonces los patrones parecen tener una relevancia parecida.

Cuando $\beta \to \infty \Rightarrow \exp \sim \infty$

entonces el patrón que se parezca más al estado actual tendrá el mayo peso. ...

Powered by CamScanner

PARCIAL_COMPUTACION_AVANZADA

October 30, 2026

0.1 PARTE COMPUTACIONAL DEL PUNTO 3 Y 4

0.2 CARLOS MARIO GARCIA ALVAREZ

0.3 1036963351

1 DE LA PARTE 3

[1]:
```
import numpy as np
import torch
import torch.nn as nn
import matplotlib.pyplot as plt

torch.set_default_dtype(torch.float64)

# Frecuencia analitica
omega = np.sqrt(2) * np.pi

# Puntos para el entrenamiento
N = 300
t_train = torch.linspace(
    0, 1, N
).reshape(-1, 1)
```

[2]:
```
def solucion_exacta(t):

    return (
        torch.cos(omega * t)
        + torch.sin(omega * t)
    )
#Aca la idea es crear el modelo A

class ModeloA(nn.Module):

    def __init__(self):

        super().__init__()

        self.A = nn.Parameter(
```

```
            torch.tensor([1.1, 0.2])
        )

        self.w = nn.Parameter(
            torch.tensor([4.2, 2.3])
        )

        self.b = nn.Parameter(
            torch.tensor([0.6, 0.1])
        )

    def forward(self, t):

        return torch.sum(
            self.A *
            torch.sin(self.w * t + self.b),
            dim=1,
            keepdim=True
        )
#luego vamos a crear el modelo B
class ModeloB(nn.Module):

    def __init__(self):

        super().__init__()

        self.A = nn.Parameter(
            torch.tensor(2**0.5)
        )

        self.w1 = nn.Parameter(
            torch.tensor(omega)
        )

        self.b1 = nn.Parameter(
            torch.tensor(0.0)
        )

        self.w2 = nn.Parameter(
            torch.tensor(1.0)
        )

        self.b2 = nn.Parameter(
            torch.tensor(np.pi / 4)
        )

    def forward(self, t):
```

```
        h1 = torch.sin(
            self.w1 * t + self.b1
        )

        return self.A * torch.sin(
            self.w2 * h1 + self.b2
        )
```

[3]:
```
#ahora la idea va a ser calcular las derivadas:

def derivadas(modelo, t):

    t = t.clone().detach().requires_grad_(True)

    a = modelo(t)

    da = torch.autograd.grad(
        a, t,
        grad_outputs=torch.ones_like(a),
        create_graph=True
    )[0]

    dda = torch.autograd.grad(
        da, t,
        grad_outputs=torch.ones_like(da),
        create_graph=True
    )[0]

    return a, da, dda
#luego se construye la funcion de perdida:

def perdida(modelo):

    # Residuo físico
    a, da, dda = derivadas(
        modelo, t_train
    )

    R = dda + 2 * np.pi**2 * a

    L_fisica = torch.mean(R**2)

    # Condiciones iniciales
    t0 = torch.zeros((1, 1))

    a0, da0, _ = derivadas(modelo, t0)
```

```
    L_IC1 = (a0 - 1)**2

    L_IC2 = (
        da0 - np.sqrt(2)*np.pi
    )**2

    L_total = (
        L_fisica
        + 10 * L_IC1.mean()
        + 10 * L_IC2.mean()
    )

    return L_total
```

#como el enunciado dice que usar uun metodo de optimizacion, voy a usar el LBFGS, que es como una alternativa al basado en gradiente

[4]:
```
def entrenar(modelo):

    optimizer = torch.optim.LBFGS(
        modelo.parameters(),
        lr=0.5,
        max_iter=2000,
        tolerance_grad=1e-12,
        tolerance_change=1e-14,
        line_search_fn="strong_wolfe"
    )

    def closure():

        optimizer.zero_grad()

        L = perdida(modelo)

        L.backward()

        return L

    optimizer.step(closure)

    return modelo

modelo_A = ModeloA()
modelo_B = ModeloB()

modelo_A = entrenar(modelo_A)
```

```
modelo_B = entrenar(modelo_B)

print("Pérdida modelo A:",
      perdida(modelo_A).item())

print("Pérdida modelo B:",
      perdida(modelo_B).item())
```

```
Pérdida modelo A: 1.3806793539759538e-14
Pérdida modelo B: 3.75770873068457e-07
```

[5]:
```
#ya teniendo una idea de la perdida de ambos modelos, podemos comparar␣
 ↪graficamente:

t_test = torch.linspace(
    0, 1, 300
).reshape(-1, 1)

a_exacta = solucion_exacta(t_test)

a_A = modelo_A(t_test)
a_B = modelo_B(t_test)

plt.figure(figsize=(10, 5))

plt.plot(
    t_test.detach().numpy(),
    a_exacta.detach().numpy(),
    label="Solución exacta",
    linewidth=2
)

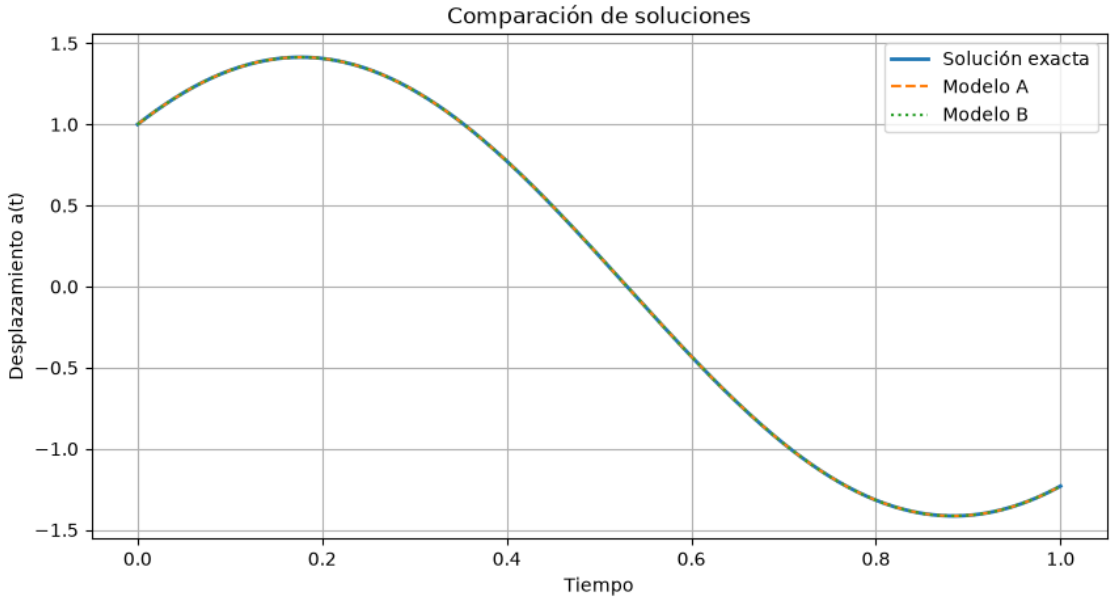
plt.plot(
    t_test.detach().numpy(),
    a_A.detach().numpy(),
    "--",
    label="Modelo A"
)

plt.plot(
    t_test.detach().numpy(),
    a_B.detach().numpy(),
    ":",
    label="Modelo B"
)

plt.xlabel("Tiempo")
```

```
plt.ylabel("Desplazamiento a(t)")
plt.title("Comparación de soluciones")

plt.legend()
plt.grid()
plt.show()
```



[6]:
```
error_A = a_A - a_exacta
error_B = a_B - a_exacta

rmse_A = torch.sqrt(
    torch.mean(error_A**2)
)

rmse_B = torch.sqrt(
    torch.mean(error_B**2)
)

max_A = torch.max(torch.abs(error_A))
max_B = torch.max(torch.abs(error_B))

print("RESULTADOS DEL ERROR")

print("RMSE A:", rmse_A.item())
print("RMSE B:", rmse_B.item())

print("Error máximo A:", max_A.item())
```

```
print("Error máximo B:", max_B.item())
```

```
RESULTADOS DEL ERROR
RMSE A: 2.2287216367814845e-08
RMSE B: 5.412844012524023e-06
Error máximo A: 3.050610231092321e-08
Error máximo B: 8.885192318008883e-06
```

[7]:
```
#Ahora vamos a analizar la conservacion de las energias:
```

[8]:
```
def energia(modelo, t):

    a, da, _ = derivadas(modelo, t)

    E = (
        (1/8)*da**2
        + (np.pi**2/4)*a**2
    )

    return E

E_A = energia(modelo_A, t_test)
E_B = energia(modelo_B, t_test)

E_exacta = np.pi**2 / 2

plt.figure(figsize=(10, 5))

plt.plot(
    t_test.detach().numpy(),
    E_A.detach().numpy(),
    label="Energía modelo A"
)

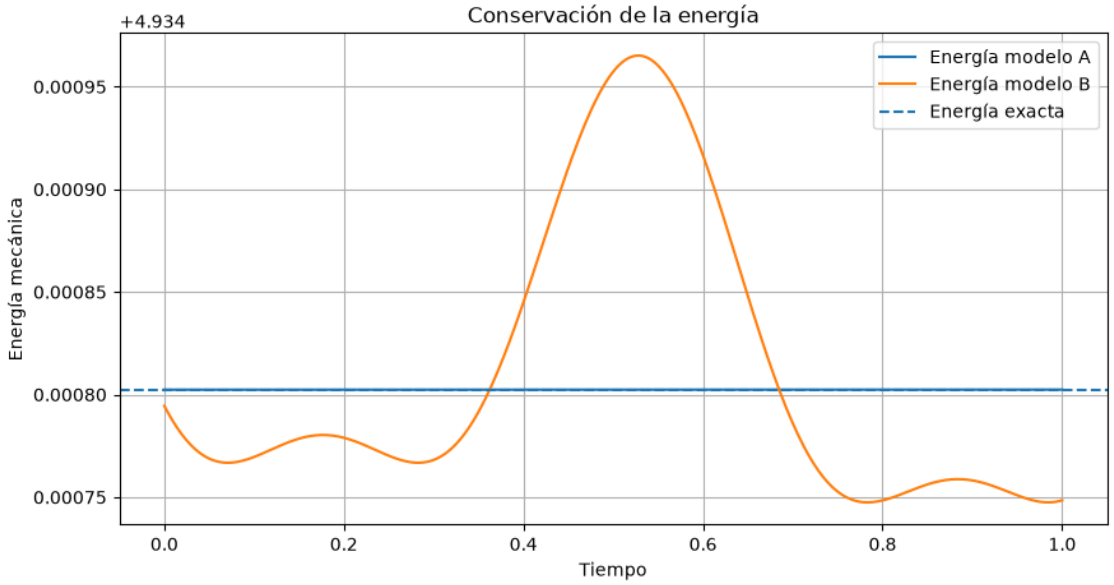
plt.plot(
    t_test.detach().numpy(),
    E_B.detach().numpy(),
    label="Energía modelo B"
)

plt.axhline(
    E_exacta,
    linestyle="--",
    label="Energía exacta"
)

plt.xlabel("Tiempo")
plt.ylabel("Energía mecánica")
```

```
plt.title("Conservación de la energía")

plt.legend()
plt.grid()
plt.show()
```



[9]:
```
def error_energia(E):

    E0 = np.pi**2 / 2

    error = torch.max(
        torch.abs(E - E0)
    )

    error_relativo = (
        error / E0
    ) * 100

    return error.item(), error_relativo.item()


dE_A, rel_A = error_energia(E_A)
dE_B, rel_B = error_energia(E_B)

print("VARIACIÓN DE ENERGÍA")

print("Modelo A:", dE_A)
```

```
print("Error relativo A (%):", rel_A)

print("Modelo B:", dE_B)
print("Error relativo B (%):", rel_B)
```

```
VARIACIÓN DE ENERGÍA
Modelo A: 1.9310492671564816e-07
Error relativo A (%): 3.9131239483992734e-06
Modelo B: 0.0001629859832359415
Error relativo B (%): 0.003302786547715163
```

1.1 conclusiones:

Al comparar las soluciones obtenidas con los dos modelos, se observa que ambas tienen un comportamiento bastante similar y se aproximan muy bien a la solución analítica, por lo que gráficamente no se aprecia una diferencia significativa entre ellas. Esto también se refleja en los errores calculados, ya que ambos son pequeños, aunque el modelo A presenta una mayor precisión, con un RMSE de 2.23 × 10^(-5), frente a 5.41 × 10^(-6) del modelo B. Sin embargo, la diferencia se hace más evidente al analizar la conservación de la energía: en la gráfica se observa que el modelo A mantiene una energía prácticamente constante, mientras que el modelo B presenta pequeñas oscilaciones. Esto se confirma con los errores relativos de energía, que son de 3.91 × 10^(-6) % para el modelo A y 0.0033 % para el modelo B. En conclusión, aunque ambos modelos ofrecen buenas aproximaciones, el modelo A presenta un mejor desempeño, ya que obtiene menores errores y conserva la energía mecánica con mayor precisión, reproduciendo mejor el comportamiento físico esperado de la membrana.

[ ]:

2 DE LA PARTE 4

[10]:
```
#basicamente dibujando las letras a mano primeramente !....
J = np.array([
    [-1,  1,  1,  1,  1,  1,  1, -1],
    [-1, -1, -1, -1, -1,  1, -1, -1],
    [-1, -1, -1, -1, -1,  1, -1, -1],
    [-1, -1, -1, -1, -1,  1, -1, -1],
    [-1, -1, -1, -1, -1,  1, -1, -1],
    [ 1, -1, -1, -1, -1,  1, -1, -1],
    [-1,  1, -1, -1,  1, -1, -1, -1],
    [-1, -1,  1,  1, -1, -1, -1, -1]
])

H = np.array([
    [ 1, -1, -1, -1, -1, -1,  1, -1],
    [ 1, -1, -1, -1, -1, -1,  1, -1],
    [ 1, -1, -1, -1, -1, -1,  1, -1],
    [ 1,  1,  1,  1,  1,  1,  1, -1],
```

```
    [ 1, -1, -1, -1, -1, -1,  1, -1],
    [ 1, -1, -1, -1, -1, -1,  1, -1],
    [ 1, -1, -1, -1, -1, -1,  1, -1],
    [ 1, -1, -1, -1, -1, -1,  1, -1]
])

X = np.array([
    J.flatten(),
    H.flatten()
])

print("Dimensiones de X:", X.shape)
```

```
Dimensiones de X: (2, 64)
```

[11]:
```
#visualizacion de los patrones
plt.figure(figsize=(8, 3))

plt.subplot(1, 2, 1)
plt.imshow(J, cmap="gray")
plt.title("Patrón J")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(H, cmap="gray")
plt.title("Patrón H")
plt.axis("off")

plt.show()
```



[12]:
```python
#ahora se crean las funciones de energia
def energia(q, X, beta):

    scores = X @ q

    maximo = np.max(beta * scores)

    energia = (
        0.5 * np.sum(q**2)
        - (maximo + np.log(
            np.sum(np.exp(beta * scores - maximo))
        )) / beta
    )

    return energia
#luego la regla de la actualizacion:
def actualizar(q, X, beta):

    scores = X @ q

    z = beta * scores

    z = z - np.max(z)

    pesos = np.exp(z) / np.sum(np.exp(z))

    q_nuevo = pesos @ X

    return q_nuevo
#despues metemos la parte iterativa
def recuperar(q_inicial, X, beta, max_iter=1000, tol=1e-6):

    q = q_inicial.copy()

    historial_energia = []

    for t in range(max_iter):

        E = energia(q, X, beta)

        historial_energia.append(E)

        q_nuevo = actualizar(q, X, beta)

        diferencia = np.linalg.norm(q_nuevo - q)

        q = q_nuevo
```

```
        if diferencia < tol:
            break

    historial_energia.append(energia(q, X, beta))

    return q, historial_energia, t + 1
```

[13]:
```
#ahora la idea es perturbar el patron:
def perturbar(patron, porcentaje):

    q = patron.copy()

    cantidad = int(len(q) * porcentaje / 100)

    indices = np.random.choice(
        len(q),
        cantidad,
        replace=False
    )

    q[indices] *= -1

    return q
#para luego intentar recuperar el patron original:
np.random.seed(13)

patron_original = X[0].copy()

q_inicial = perturbar(patron_original, 20)

beta = 1.0

q_final, historial, iteraciones = recuperar(
    q_inicial, X, beta
)

print("Iteraciones:", iteraciones)
```

```
Iteraciones: 2
```

[14]:
```
#convirtiendo a binario
q_binario = np.where(q_final >= 0, 1, -1)
#y calculamos el porcentaja de recuperacion:
precision = np.mean(
    q_binario == patron_original
) * 100
```

```
print("Porcentaje de recuperación:", precision, "%")
```

```
Porcentaje de recuperación: 100.0 %
```

[15]:
```
# al final mostramos los patrones orginales, perturbados y recuperados:

plt.figure(figsize=(12, 6))

imagenes = [
    patron_original.reshape(8, 8),
    q_inicial.reshape(8, 8),
    q_final.reshape(8, 8),
    q_binario.reshape(8, 8)
]

titulos = [
    "Patrón original",
    "Patrón perturbado",
    "Estado final continuo",
    "Estado final binario"
]

for i in range(4):

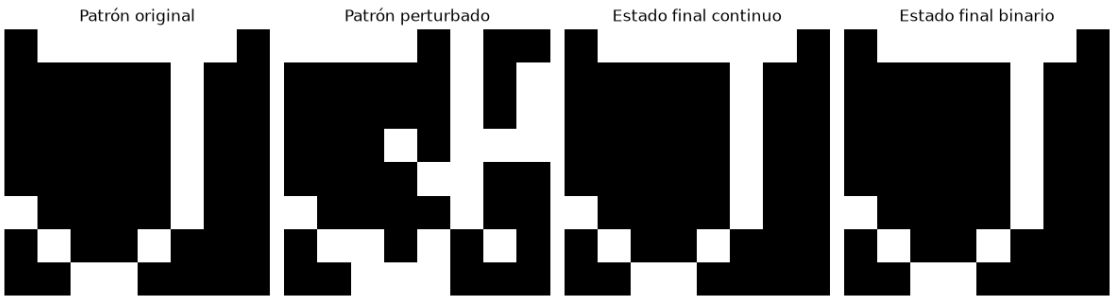
    plt.subplot(1, 4, i + 1)

    plt.imshow(imagenes[i], cmap="gray", vmin=-1, vmax=1)

    plt.title(titulos[i])

    plt.axis("off")

plt.tight_layout()
plt.show()
```



3 ahora lo que se quiere estudiar es como lo afecta la perturbacion... para ver que tan dificil se pone la recuperacion

[16]:
```
porcentajes = [5, 10, 20, 30, 40, 50]

resultados = []

beta = 1.0

for porcentaje in porcentajes:

    precisiones = []

    for prueba in range(50):

        q_inicial = perturbar(X[0], porcentaje)

        q_final, historial, iteraciones = recuperar(
            q_inicial, X, beta
        )

        q_binario = np.where(q_final >= 0, 1, -1)

        precision = np.mean(
            q_binario == X[0]
        ) * 100

        precisiones.append(precision)

    promedio = np.mean(precisiones)

    resultados.append(promedio)

    print(
        f"Perturbación: {porcentaje}% "
        f"- Recuperación: {promedio:.2f}%"
    )
```

```
Perturbación: 5% - Recuperación: 100.00%
Perturbación: 10% - Recuperación: 100.00%
Perturbación: 20% - Recuperación: 100.00%
Perturbación: 30% - Recuperación: 100.00%
Perturbación: 40% - Recuperación: 96.12%
Perturbación: 50% - Recuperación: 76.75%
```

[17]:
```
#graficamos lo que acabamos de ver:
```

```
plt.figure(figsize=(8, 5))

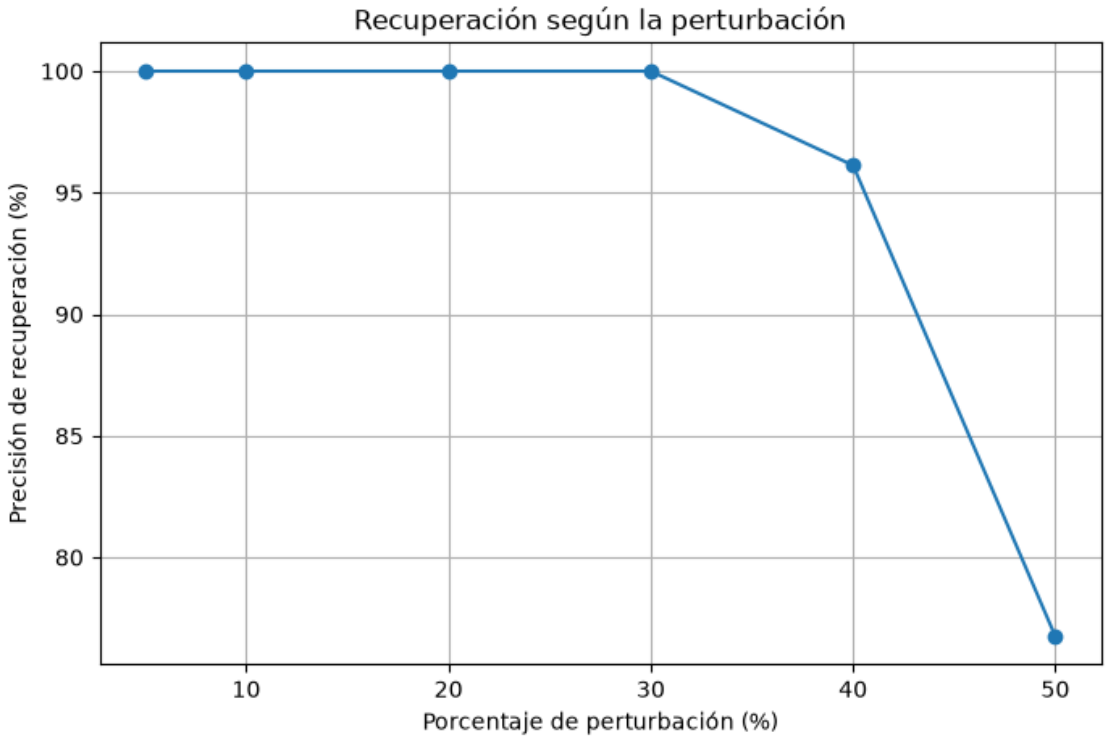
plt.plot(
    porcentajes,
    resultados,
    marker="o"
)

plt.xlabel("Porcentaje de perturbación (%)")
plt.ylabel("Precisión de recuperación (%)")

plt.title("Recuperación según la perturbación")

plt.grid()

plt.show()
```



4 ahora lo que se va a analizar es el efecto de “betha”

[18]:
```
#se van a usar el mismo patron perturbado para comprar los resultados de mejor␣
 ↪manera:

valores_beta = [0.1, 0.5, 1, 2, 5, 10]

q_inicial = perturbar(X[0], 20)

precisiones_beta = []

plt.figure(figsize=(9, 5))

for beta in valores_beta:

    q_final, historial, iteraciones = recuperar(
        q_inicial, X, beta
    )

    q_binario = np.where(q_final >= 0, 1, -1)

    precision = np.mean(
        q_binario == X[0]
    ) * 100

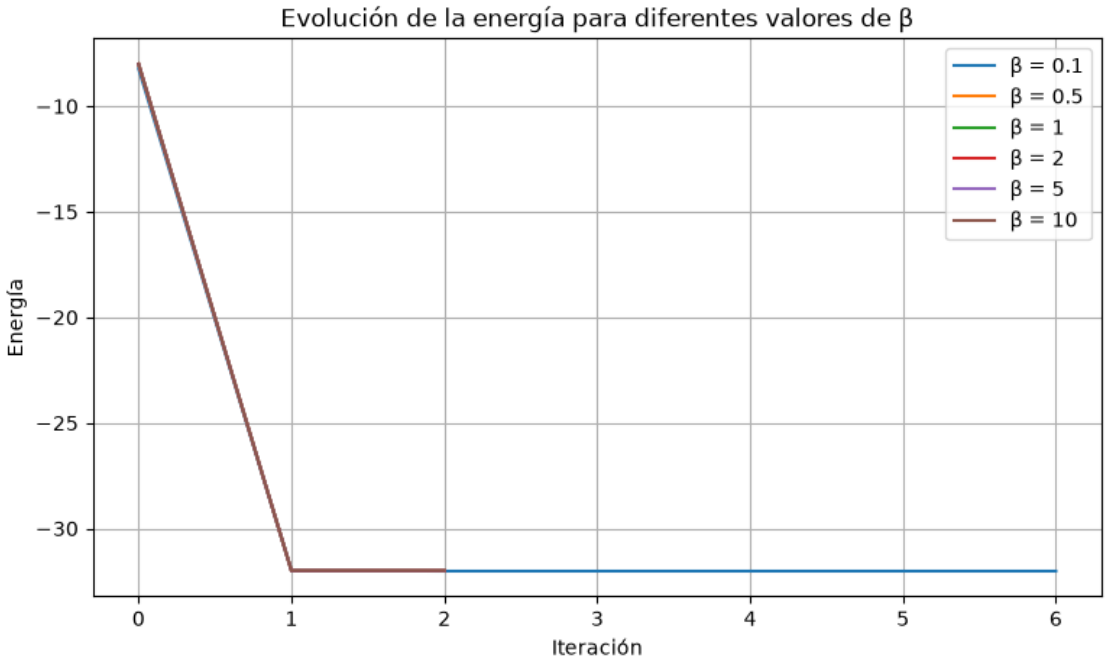
    precisiones_beta.append(precision)

    plt.plot(
        historial,
        label=f"  = {beta}"
    )

plt.xlabel("Iteración")
plt.ylabel("Energía")

plt.title("Evolución de la energía para diferentes valores de  ")

plt.legend()
plt.grid()
plt.show()
```



[19]:
```
#luego vemos los porcentajes de recuperacion:

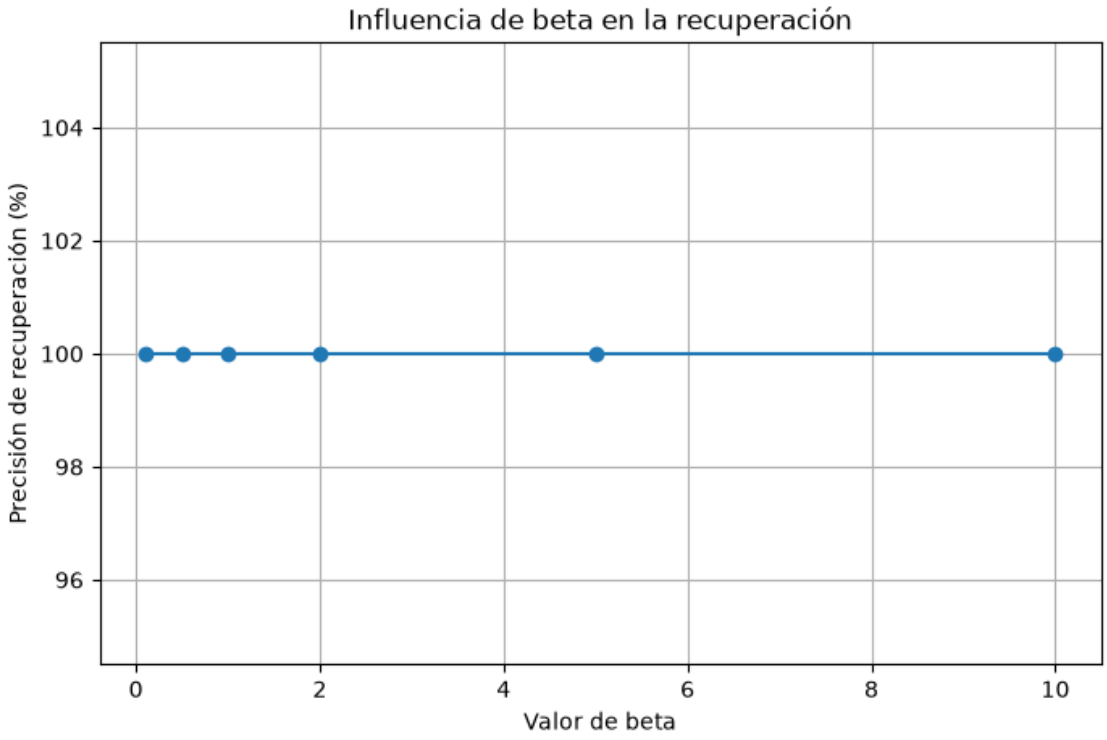
plt.figure(figsize=(8, 5))

plt.plot(
    valores_beta,
    precisiones_beta,
    marker="o"
)

plt.xlabel("Valor de beta")
plt.ylabel("Precisión de recuperación (%)")

plt.title("Influencia de beta en la recuperación")

plt.grid()
plt.show()
```



5 CONCLUSIONES

1. Al realizar la recuperación de la letra J con una perturbación del 20 %, la red logró recuperar el patrón original con un 100 % de precisión. Además, solamente necesitó 2 iteraciones para llegar al resultado, lo que demuestra que pudo recuperar la información rápidamente.

2. Al aumentar el porcentaje de píxeles modificados, observamos que la recuperación se vuelve más difícil. Con perturbaciones del 5 %, 10 %, 20 % y 30 % se obtuvo un 100 % de recuperación. Sin embargo, con un 40 % la recuperación bajó a 96,12 %, y con un 50 % disminuyó a 76,75 %. Esto nos muestra que mientras más se dañe el patrón original, más difícil es para la red reconocerlo correctamente.

3. Al analizar los diferentes valores de , la última gráfica mostró un 100 % de recuperación para todos los valores probados. Esto significa que, en nuestro experimento, cambiar no afectó el resultado final de la recuperación binaria. Sin embargo, este parámetro sí influye en la importancia que la red le da a cada patrón almacenado durante el proceso de recuperación.

4. En general, podemos concluir que la memoria asociativa funciona correctamente cuando el patrón recibido tiene pocas modificaciones. Sin embargo, cuando la cantidad de píxeles perturbados aumenta demasiado, la red empieza a tener dificultades para recuperar la letra original.

5.1 FIN

[ ]: